

Linux Server Operations Lab

基礎編 最終レビュー

正常状態の確認から、アップデート後の総合点検まで

対象環境: Ubuntu Server 24.04 / tokyo 192.168.56.11 / controller 192.168.56.10

目的: Linuxサーバの状態を、コマンドを実行して確認し、出力から「正常そうか」を判断できるようになる。自動化の前に、手動運用の確認手順と判断軸を身につける。

作成日: 2026-10-03

1. 基礎編のゴール

この基礎編では、単にLinuxコマンドを暗記するのではなく、「何を確認したいのか → どのコマンドを使うのか → 出力のどこを見るのか → 現在の状態をどう判断するのか」という流れを実機で確認した。

最終総合演習では、前日にパッケージアップデートと再起動を実施した tokyo サーバについて、「正常に稼働しているか一通り点検する」という想定で確認を行った。

確認分野	主な確認内容
基本リソース	ディスク、メモリ、CPU、負荷、プロセス状態
Webサービス	nginx、プロセス、80番ポート、HTTP応答
ネットワーク	IPアドレス、疎通、名前解決
ログ	systemd/journalによるサービス履歴
ユーザー・権限	whoami、UID/GID、グループ、sudo
SSH	サービス、22番ポート、公開鍵認証、セッション
更新後確認	カーネル、パッケージ更新状態、再起動後のサービス

2. 実機環境

ホスト	IP	役割
controller	192.168.56.10	管理・接続元
tokyo	192.168.56.11	今回の点検対象
yokohama	192.168.56.12	既存の検証対象
omiya	192.168.56.13	既存の検証対象

tokyo には VirtualBox 上で複数のネットワークインターフェースがあり、10.0.2.15 は外部側、192.168.56.11 は検証環境内の controller などとの通信に使用している。127.0.0.1 / localhost はサーバ自身を指すループバックである。

3. 正常状態を確認する基本の考え方

確認では「設定されている」とことと「実際に動作している」ことを分けて考えた。特にWebサービスでは、次のように段階的に確認した。

サービス状態 → プロセス → 待受ポート → 実際のHTTP通信 → ログ

ネットワークでも同様に、IP設定だけで終わらず、実際の疎通と名前解決まで確認した。

IPアドレス → 経路/疎通 → DNS設定 → 実際の名前解決

4. ディスク容量の確認

```
df -h
```

総合演習では /dev/sda2 のルート領域が Size 25G、Used 6.2G、Avail 18G、Use% 27% だった。現時点で容量逼迫を示す状態ではないと判断した。

重要な見方は、Used だけでなく Use% と Mounted on を合わせて確認し、「どの領域がどの程度使われているか」を見ること。

```
[tokyo@tokyo:~$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	97M	1.2M	95M	2%	/run
/dev/sda2	25G	6.2G	18G	27%	/
tmpfs	481M	0	481M	0%	/dev/shm
tmpfs	5.0M	0	5.0M	0%	/run/lock
tmpfs	481M	0	481M	0%	/run/qemu
/dev/sdb	2.0G	24K	1.8G	1%	/data
/dev/mapper/vg_data-lv_data	1.5G	28K	1.4G	1%	/lvmdata
tmpfs	97M	16K	97M	1%	/run/user/1000

図: df -h によるディスク確認

5. メモリの確認

```
free -h
```

結果は total 961Mi、used 329Mi、free 365Mi、available 631Mi、Swap used 0B。available が十分残っており、現時点でメモリ逼迫を示す状態ではない。

Linuxではキャッシュにもメモリを利用するため、「free が少ない = 直ちにメモリ不足」とは判断しない。available を重要な判断材料として見る。

```
[tokyo@tokyo:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	961Mi	329Mi	365Mi	1.1Mi	408Mi	631Mi
Swap:	511Mi	0B	511Mi			

図: free -h によるメモリ確認

6. CPU・負荷の確認

```
top
```

総合演習では load average 0.00, 0.00, 0.02、CPU idle 99.6%、zombie 0。CPUはほぼアイドル状態で、負荷も低いと判断した。

id は idle の略で、「CPUを使っている割合」ではなく「CPUが空いている割合」と理解する。

```

top - 00:16:11 up 29 min, 1 user, load average: 0.00, 0.00, 0.02
Tasks: 112 total, 1 running, 111 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 99.6 id, 0.0 wa, 0.0 hi, 0.4 si, 0.0 st
MiB Mem : 961.6 total, 366.0 free, 328.8 used, 409.4 buff/cache
MiB Swap: 512.0 total, 512.0 free, 0.0 used. 632.7 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
 1630 root        20   0       0       0       0 I   0.7   0.0   0:04.84 kworker/0:0-events
 1608 tokyo      20   0   14996    7112   5152 S   0.3   0.7   0:00.41 sshd
    1 root        20   0   22320   13612   9576 S   0.0   1.4   0:04.61 systemd
    2 root        20   0       0       0       0 S   0.0   0.0   0:00.01 kthreadd
    3 root        20   0       0       0       0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
    5 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
    6 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-slub_
    7 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-netns
   11 root        20   0       0       0       0 I   0.0   0.0   0:01.06 kworker/u2:0-events_pow+
   12 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
   13 root        20   0       0       0       0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
   14 root        20   0       0       0       0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
   15 root        20   0       0       0       0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
   16 root        20   0       0       0       0 S   0.0   0.0   0:00.44 ksoftirqd/0
   17 root        20   0       0       0       0 I   0.0   0.0   0:00.33 rcu_preempt
   18 root        rt    0       0       0       0 S   0.0   0.0   0:00.04 migration/0
   19 root       -51   0       0       0       0 S   0.0   0.0   0:00.00 idle_inject/0
   20 root        20   0       0       0       0 S   0.0   0.0   0:00.00 cpuhp/0
   21 root        20   0       0       0       0 S   0.0   0.0   0:00.01 kdevtmpfs
   22 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-inet_
   23 root        20   0       0       0       0 S   0.0   0.0   0:00.00 kauditd
   24 root        20   0       0       0       0 S   0.0   0.0   0:00.00 khungtaskd
   25 root        20   0       0       0       0 I   0.0   0.0   0:00.60 kworker/u2:1-events_pow+
   26 root        20   0       0       0       0 S   0.0   0.0   0:00.00 oom_reaper
   28 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-write
   29 root        20   0       0       0       0 S   0.0   0.0   0:00.21 kcompactd0
   30 root        25   5       0       0       0 S   0.0   0.0   0:00.00 ksmd
   31 root        39  19       0       0       0 S   0.0   0.0   0:00.00 khugepaged
   32 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-kinte
   33 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-kbloc
   34 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-blkcg
   35 root       -51   0       0       0       0 S   0.0   0.0   0:00.00 irq/9-acpi
   36 root        0 -20     0       0       0 I   0.0   0.0   0:00.00 kworker/R-tpm_d

```

図: top によるCPU・負荷・プロセス確認

7. nginx - サービスからHTTPまで

7.1 サービス状態

```
systemctl status nginx
```

Active: active (running) から現在稼働中、enabled からOS起動時の自動起動が有効と確認した。active と enabled は別の意味である。

```
[tokyo@tokyo:~$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-10-02 23:46:53 UTC; 1h 0min ago
     Docs: man:nginx(8)
  Process: 690 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exit>
 Process: 752 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, statu>
 Main PID: 790 (nginx)
    Tasks: 2 (limit: 1055)
   Memory: 3.0M (peak: 3.3M)
      CPU: 390ms
   CGroup: /system.slice/nginx.service
           └─790 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─794 "nginx: worker process"

Oct 02 23:46:48 tokyo systemd[1]: Starting nginx.service - A high performance web server and a >
Oct 02 23:46:53 tokyo systemd[1]: Started nginx.service - A high performance web server and a r>
lines 1-16/16 (END)
```

図: nginx の systemctl status

7.2 80番ポート

```
sudo ss -lntp | grep ':80'
```

0.0.0.0:80 と [::]:80 が LISTEN し、nginx プロセスが利用していることを確認した。

```
[tokyo@tokyo:~$ sudo ss -lntp | grep ':80'
[sudo] password for tokyo:
LISTEN 0      511          0.0.0.0:*      0.0.0.0:*      users: (("nginx",pid=794,fd=5),("nginx",pid=790,fd=5))
LISTEN 0      511          [::]:*        [::]:*        users: (("nginx",pid=794,fd=6),("nginx",pid=790,fd=6))
tokyo@tokyo:~$
```

図: nginx の80番ポート待受

7.3 HTTP応答

```
curl -I http://localhost
```

HTTP/1.1 200 OK

が返り、nginxが起動しているだけでなく、実際にHTTPリクエストへ正常応答できることを確認した。

```
[tokyo@tokyo:~$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 03 Oct 2026 00:57:48 GMT
Content-Type: text/html
Content-Length: 454
Last-Modified: Sun, 13 Sep 2026 06:19:58 GMT
Connection: keep-alive
ETag: "6aa6408e-1c6"
Accept-Ranges: bytes
```

図: localhost へのHTTP確認

8. ネットワークと名前解決

8.1 IPアドレス

```
ip addr
```

lo の 127.0.0.1 は自分自身を表すループバック。enp0s3 の 10.0.2.15 は外部側、別インターフェースの 192.168.56.11 は controller など検証環境内の通信に利用している。

```
tokyo@tokyo:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:4c:c2:3c brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86337sec preferred_lft 86337sec
    inet6 fd17:625c:f037:2:a00:27ff:fe4c:c23c/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86340sec preferred_lft 14340sec
```

図: ip addr の一部

8.2 controllerへの疎通

```
ping -c 4 192.168.56.10
```

4 packets transmitted, 4 received, 0% packet loss。tokyo から controller への疎通は正常と判断した。

```
valid_lft forever preferred_lft forever
tokyo@tokyo:~$ ping -c 4 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=1.56 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=0.671 ms
64 bytes from 192.168.56.10: icmp_seq=3 ttl=64 time=26.2 ms
64 bytes from 192.168.56.10: icmp_seq=4 ttl=64 time=1.55 ms

--- 192.168.56.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3045ms
rtt min/avg/max/mdev = 0.671/7.494/26.196/10.803 ms
```

図: controllerへのping

8.3 名前解決

```
getent hosts example.com
```

example.com に対してIPv6アドレスが返り、名前解決が成功した。IPv6が返ること自体は異常ではない。IP疎通とDNS名前解決は別々に確認できることを再確認した。

```
tokyo@tokyo:~$ getent hosts example.com
2606:4700:10::ac42:93f3 example.com
2606:4700:10::6814:179a example.com
```

図: getent hosts による名前解決

9. ログの確認

```
sudo journalctl -u nginx -n 20 --no-pager
```

nginx の Starting / Started、過去の Stopping / Deactivated successfully / Stopped などを確認した。過去に停止履歴があるだけで、現在の障害とは判断しない。時刻、原因、現在状態、実際の通信を合わせて判断する。

入力時に --no-pager[と余計な [を付け、unrecognized option

エラーが発生した。エラーメッセージから指定オプションを確認し、余計な文字を削除して再実行した。

```
tokyo@tokyo:~$ sudo journalctl -u nginx -n 20 --no-pager[
[sudo] password for tokyo:
journalctl: unrecognized option '--no-pager['
tokyo@tokyo:~$
```

図: 誤入力による journalctl エラー

```
[tokyo@tokyo:~$ sudo journalctl -u nginx -n 20 --no-pager
Sep 27 22:30:01 tokyo systemd[1]: Started nginx.service - A high performance web
server and a reverse proxy server.
-- Boot 3ea2db94a9124334961a70d6c37c8da0 --
Sep 29 12:01:37 tokyo systemd[1]: Starting nginx.service - A high performance we
b server and a reverse proxy server...
Sep 29 12:01:41 tokyo systemd[1]: Started nginx.service - A high performance web
server and a reverse proxy server.
Sep 29 12:24:39 tokyo systemd[1]: Stopping nginx.service - A high performance we
b server and a reverse proxy server...
Sep 29 12:24:39 tokyo systemd[1]: nginx.service: Deactivated successfully.
Sep 29 12:24:39 tokyo systemd[1]: Stopped nginx.service - A high performance web
server and a reverse proxy server.
Sep 29 12:24:39 tokyo systemd[1]: Starting nginx.service - A high performance we
b server and a reverse proxy server...
Sep 29 12:24:39 tokyo systemd[1]: Started nginx.service - A high performance web
server and a reverse proxy server.
-- Boot 7f9e392f4f474338604a992fc708a5d --
Oct 02 12:05:22 tokyo systemd[1]: Starting nginx.service - A high performance we
b server and a reverse proxy server...
```

図: 修正後の nginx journal

10. ユーザー・グループ・sudo

10.1 現在のユーザー

```
whoami
# tokyo
```

hostname はサーバ名、whoami は現在操作しているユーザーを確認する。

```
tokyo@tokyo:~$ whoami
tokyo
```

図: whoami

10.2 UID / GID / 所属グループ

id

uid=1000(tokyo)、gid=1000(tokyo)、groups に 27(sudo) などが含まれることを確認。UIDはUser ID、GIDはGroup ID。

```
[tokyo@tokyo:~$ id
uid=1000(tokyo) gid=1000(tokyo) groups=1000(tokyo),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),101(lxd),111(libvirt),113(docker)
```

図: id によるユーザー情報

10.3 sudo権限

```
sudo -l
```

(ALL : ALL) ALLに加え、(root) NOPASSWD: /usr/bin/systemctl restart nginx が設定されていることを確認した。後者はnginx再起動に限りパスワードなしでroot権限実行できる設定である。

```
tokyo@tokyo:~$ sudo -l
```

Matching Defaults entries for tokyo on tokyo:

```
env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
use_pty
```

User tokyo may run the following commands on tokyo:

```
(ALL : ALL) ALL
```

```
(root) NOPASSWD: /usr/bin/systemctl restart nginx
```

図: sudo -l

11. SSHの確認

11.1 サービス

```
systemctl status ssh
```

SSHは active (running)、enabled。ログ部分から 0.0.0.0 port 22 / :: port 22 で待ち受け、Accepted publickey と session opened も確認できた。

```
[tokyo@tokyo:~$ systemctl status ssh
```

```
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-10-03 02:08:25 UTC; 48min ago
 TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
   Process: 781 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 847 (sshd)
    Tasks: 1 (limit: 1055)
   Memory: 5.1M (peak: 5.7M)
      CPU: 742ms
   CGroup: /system.slice/ssh.service
           └─847 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

```
Oct 03 02:08:17 tokyo systemd[1]: Starting ssh.service - OpenBSD Secure Shell s>
Oct 03 02:08:25 tokyo sshd[847]: Server listening on 0.0.0.0 port 22.
Oct 03 02:08:25 tokyo sshd[847]: Server listening on :: port 22.
Oct 03 02:08:25 tokyo systemd[1]: Started ssh.service - OpenBSD Secure Shell se>
Oct 03 02:09:05 tokyo sshd[1282]: Accepted publickey for tokyo from 192.168.56.>
Oct 03 02:09:05 tokyo sshd[1282]: pam_unix(sshd:session): session opened for us>
```

図: SSHサービス状態

11.2 22番ポート

```
sudo ss -ltnp | grep ':22'
```

sshd がIPv4/IPv6の22番ポートで LISTEN していることを確認した。80番確認で使ったコマンドを22番へ応用できた。

```
tokyo@tokyo:~$ sudo ss -lntp | grep ':22'
[sudo] password for tokyo:
LISTEN 0      4096      0.0.0.0:22      0.0.0.0:*      users: (("sshd",pid=84
7,fd=3),("systemd",pid=1,fd=96))
LISTEN 0      4096      [::]:22      [::]:*      users: (("sshd",pid=84
7,fd=4),("systemd",pid=1,fd=97))
```

図: SSH 22番ポート

12. パッケージアップデートと再起動後確認

前日の学習では `apt list --upgradable → apt update → apt-get -s upgrade → apt upgrade`

の順で、更新候補、パッケージ情報更新、シミュレーション、実更新を確認した。apt upgrade 中には keyboard-configuration の対話画面も発生し、自動化時には対話待ちが問題になり得ることを学んだ。

更新後に `/var/run/reboot-required` を確認すると再起動が必要となり、再起動前のカーネル 6.8.0-139-generic から、再起動後 6.8.0-146-generic へ切り替わった。再起動後には nginx、80番ポート、HTTP 200 まで再確認した。

12.1 総合演習でのカーネル確認

```
uname -r
# 6.8.0-146-generic
```

```
[tokyo@tokyo:~$ uname -r
6.8.0-146-generic
```

図: 現在のカーネル

12.2 パッケージ状態

本来は状態確認として `apt list --upgradable` を実行する想定だったが、総合演習では `sudo apt upgrade` を実行した。結果は 0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded で、実際の変更は発生しなかった。9件は phased updates により保留された。

「確認だけしたい」ときに変更系コマンドを実行しないことも運用上の重要なポイント。不要パッケージについて autoremove の案内も出たが、総合点検では不要な変更を避け、実行しなかった。

```
tokyo@tokyo:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libgusb2 linux-headers-6.8.0-138 linux-headers-6.8.0-138-generic
  linux-image-6.8.0-138-generic linux-modules-6.8.0-138-generic
  linux-modules-extra-6.8.0-138-generic linux-tools-6.8.0-138
  linux-tools-6.8.0-138-generic
Use 'sudo apt autoremove' to remove them.
The following upgrades have been deferred due to phasing:
  alsa-ucm-conf dnsmasq-base libgbm1 libgl1-mesa-dri libglx-mesa0
  mesa-libgallium mesa-va-drivers mesa-vulkan-drivers thermald
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.
```

図: apt upgrade 実行結果

13. 最終総合点検の判定

項目	結果	判定
ディスク	/ 使用率 27%、空き18G	正常そう
メモリ	available 631Mi、Swap使用0B	逼迫なし
CPU/負荷	idle 99.6%、load 0.00/0.00/0.02	低負荷
nginx	active (running)、enabled	稼働
HTTP	80 LISTEN、HTTP/1.1 200 OK	正常応答
ネットワーク	controllerへ4/4、0% loss	疎通OK
DNS	example.comの名前解決成功	名前解決OK
ログ	起動/停止履歴を確認、現在の異常材料なし	要文脈判断
ユーザー	tokyo、uid/gid 1000	確認OK
sudo	sudo権限と個別NOPASSWD設定を確認	確認OK
SSH	active、22 LISTEN、公開鍵認証成功	接続OK
カーネル	6.8.0-146-generic	更新後で稼働
パッケージ	今回のupgradeは変更0、9件phasing保留	状態確認済み

結論: 今回確認した範囲では、tokyoサーバに明確な異常は見つからなかった。ただし「正常」は永久的な保証ではなく、点検時点の観測結果に基づく判断である。

14. 総合演習で分かった定着度

分類	内容
自力で出てきた	df、free、top、ss、ip、ping。idle の意味からCPU状態を判断。80番確認を22番へ応用。
うろ覚え・ヒントで復元	df -h / free -h の -h、systemctl status、curl -I http://localhost。
重点復習	getent hosts、journalctl、whoami、id、sudo -l、uname -r、apt list --upgradable。
理解面で修正した点	freeとavailable、activeとenabled、127.0.0.1/localhost、確認コマンドと変更コマンドの区別。

今回の傾向は、「何を確認するか」「出力からどう判断するか」は徐々に身につけている一方、コマンド名やオプションを即座に取り出す部分は復習が必要、というものだった。

15. 復習用チートシート

目的	コマンド	見るポイント
ディスク	df -h	Use%、Mounted on、Avail
メモリ	free -h	available、used、Swap
CPU/負荷	top	load average、id、wa、zombie
サービス	systemctl status nginx	active / enabled / PID
ポート	sudo ss -lntp grep ':80'	LISTEN、プロセス
HTTP	curl -I http://localhost	HTTP 200など
IP	ip addr	対象NIC、inet、UP/DOWN
疎通	ping -c 4 IP	received、packet loss
名前解決	getent hosts example.com	名前からIPが返るか
ログ	sudo journalctl -u nginx -n 20 --no-pager	時刻、Starting/Started、エラー
現在ユーザー	whoami	操作ユーザー
UID/GID	id	uid、gid、groups
sudo権限	sudo -l	許可コマンド
SSH	systemctl status ssh	active、22、認証ログ
カーネル	uname -r	現在起動中のkernel
更新候補	apt list --upgradable	確認のみ。変更しない

16. LinuCレベル1の知識との接続

今回の実機学習では、ファイルシステム、プロセス、システムリソース、systemd、ネットワーク、DNS、ユーザー/グループ、権限、SSH、パッケージ管理、ログなど、LinuCレベル1で学ぶLinuxの基礎知識を「実際のサーバを点検する」という形でつなげた。

資格学習の用語やコマンドを、単独の暗記ではなく「サーバが正常か確認するためにどう使うか」という目的と結びつけたことが、このラボの基礎編での中心的な学びである。

17. 自動化候補

基礎編では自動化そのものより、まず手動で意味を理解することを優先した。そのうえで、繰り返し確認する項目は次の段階で自動化候補になる。

手段	候補
Ansible	複数サーバの日次点検、サービス状態、ポート/HTTP確認、更新前後確認
Python	コマンド結果の解析、OK/WARNING判定、異常値の抽出
Excel	サーバ台帳、点検結果、履歴、異常項目の一覧化
AWX（後段）	定型ジョブの実行、結果確認、運用フロー化

18. GitHub・noteへ残すときの材料

GitHubには、実行手順・確認コマンド・結果・学び・スクリーンショットを整理し、単なるコマンド集ではなく「なぜ確認するか」が分かる形で残す。

noteでは、無料記事は1テーマ1解決型として、日次点検、ユーザー/権限、アップデート後確認などを切り出せる。有料の基礎編では、複数の確認を一連の運用点検として体系化し、判断軸と失敗・修正も含めると差別化しやすい。

今回の総合演習そのものも、「覚えていたコマンド／忘れていたコマンド」「出力は読めたか」という振り返りを含めることで、学習記録として記事化できる。

19. 次の段階

基礎編では「正常状態を確認する」力を中心に学んだ。次の発展編では、意図的に安全な障害を発生させ、症状確認 → 情報収集 → 原因切り分け → 手動修正 → 復旧確認 → 再発防止 → 自動化候補、という流れを実機で経験する。

正常状態
↓
障害発生
↓
症状確認
↓
情報収集
↓
原因切り分け
↓
修正
↓
復旧確認
↓
再発防止
↓
自動化候補
基礎編 完了。